

Deep Learning

Lecture 9: Variational and Implicit Models

Amir Atapour-Abarghouei

amir.atapour-abarghouei@durham.ac.uk

Durham University





1 Divergence measures

- Kullback–Leibler divergence

2 Variational autoencoders

- Autoencoders
- VAEs
- Reparameterisation trick
- ELBO

3 Implicit networks

- Definition
- SIRENs
- NeRFs

Autoencoders



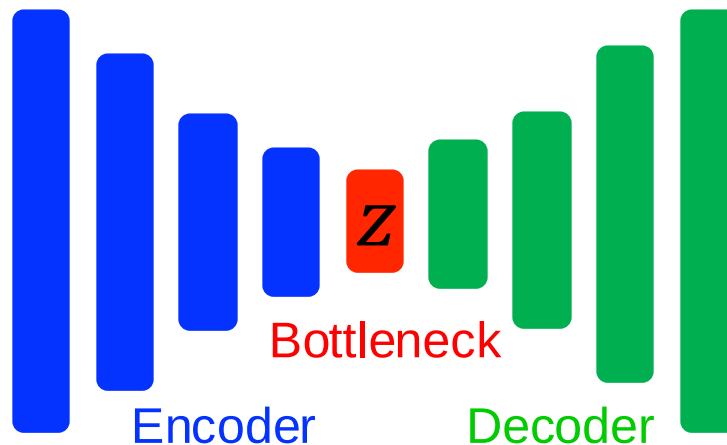
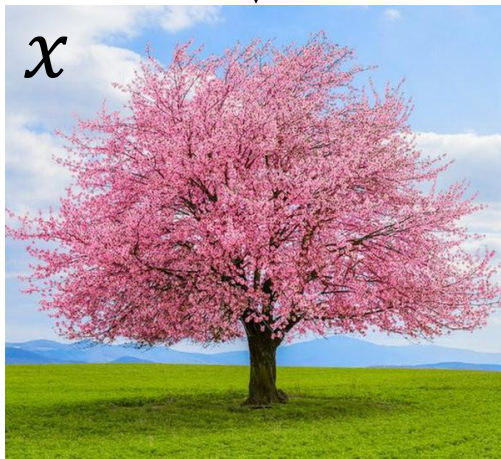
- **Not** generative models.
- Designed to obtain latent variables.
- Latent variables inferred by an autoencoder are a lower dimensional feature representation from unlabeled data.

variables that are not directly observed but are rather inferred (through a model)

L2 Loss: sum of the squared differences

$$\mathcal{L}(x, \hat{x}) = \|x - \hat{x}\|^2$$

reconstruct the input!



Autoencoders

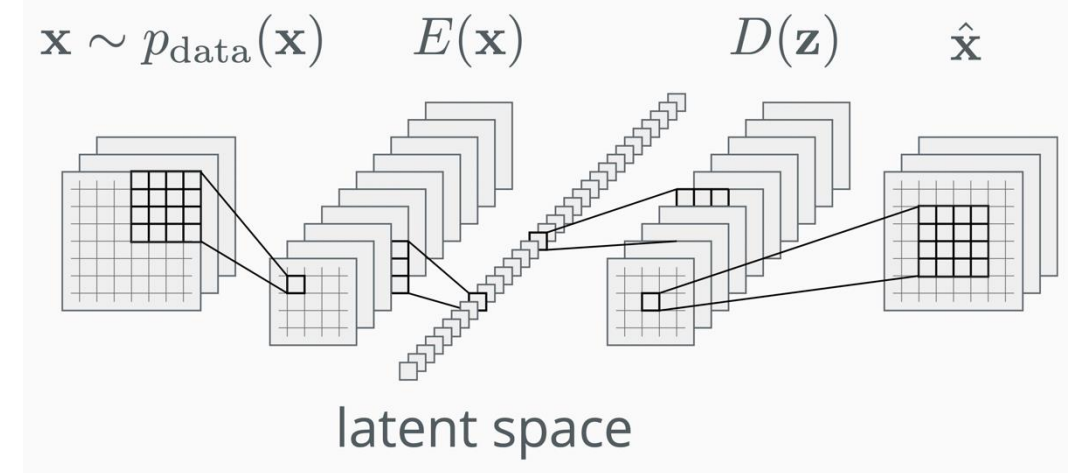


Definition: Autoencoder

An autoencoder is a feedforward neural network that reconstructs its inputs, thus learning an identity:

$$\mathcal{L}_{AE} = E_{x \sim p_{\text{data}}} [\mathcal{L}(x, D(E(x)))]$$

where $\mathcal{L}$ is a loss function. The encoder function $E : R_n \rightarrow R_m$ compresses the dimensionality of the data $n \ll m$ to a latent encoding $z = E(x)$, which is then recovered by the decoder $D : R_m \rightarrow R_n$, where $\hat{x} = D(z)$.

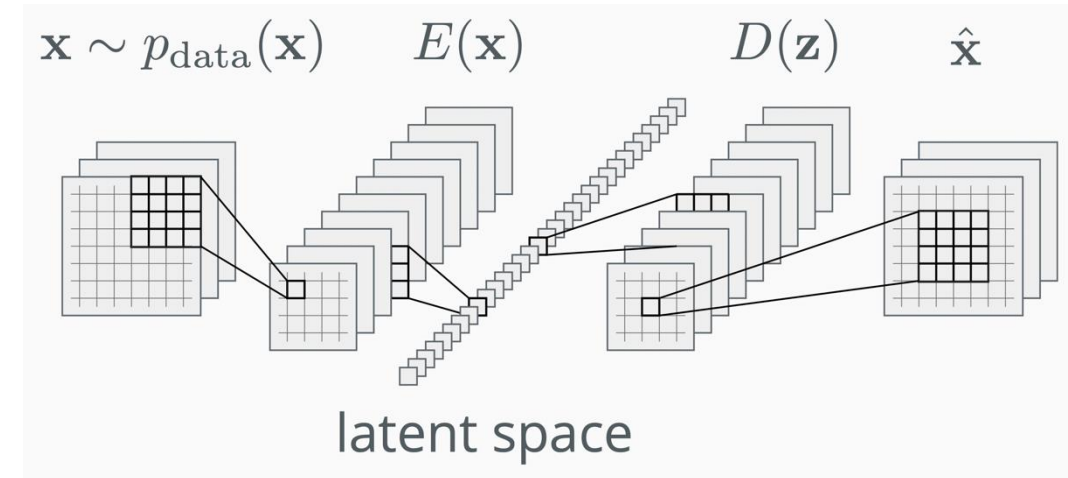


Autoencoders



Properties: Autoencoder

- Very good at representation learning.
- Are being used for data compression.
- Denoising Autoencoders
 - ↪ Reconstruct image from a noisy input.
- Inpainting Autoencoders
 - ↪ Reconstruct image from a input with missing regions.



Let's Look at Some Code!



Example of a Simple Autoencoder

Code on GitHub: https://github.com/atapour/dl-pytorch/blob/main/AutoEncoder_Example/AutoEncoder_Example.ipynb

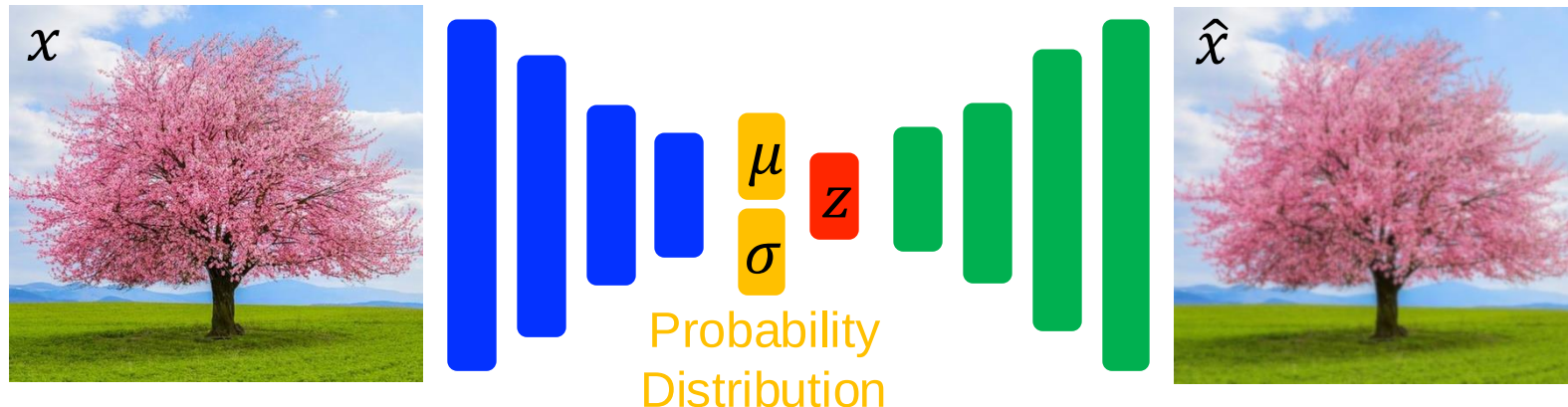
Code on Colab: https://colab.research.google.com/github/atapour/dl-pytorch/blob/main/AutoEncoder_Example/AutoEncoder_Example.ipynb

Variational Autoencoders (VAE)

[Kingma and Welling, 2013]



- Explicit approximate variational density function.
- **Generative Models** with a probabilistic spin on autoencoders.
- Instead of a fixed latent vector (as in an autoencoder), **VAEs learn a probability distribution** in the latent space that facilitates sampling new data for generation.



Variational Autoencoders (VAE)

[Kingma and Welling, 2013]



- The encoder learns:

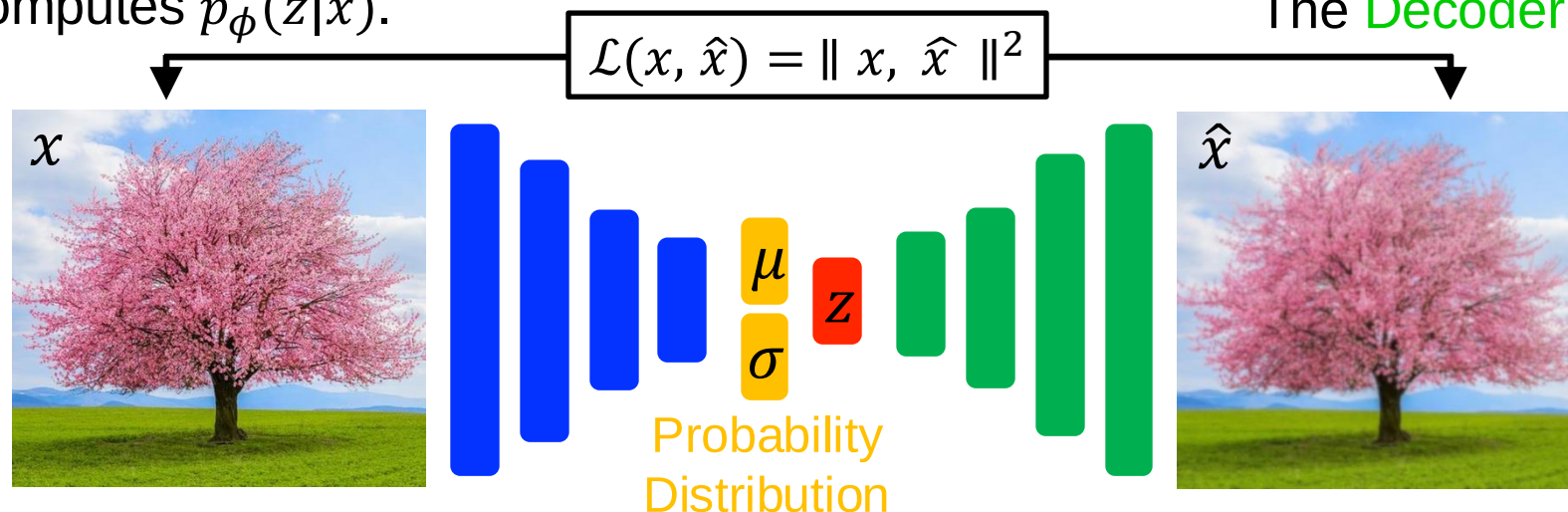
- vector μ (**means**)
- vector σ (**standard deviations**)



- The loss function will include a reconstruction loss, just like the vanilla autoencoder, but there is more

The **Encoder** computes $p_\phi(z|x)$.

The **Decoder** computes $q_\theta(x|z)$.



Variational Autoencoders (VAE)

[Kingma and Welling, 2013]



- The loss function consists of two terms:

- Reconstruction Loss (L_2 loss)

- KL Divergence

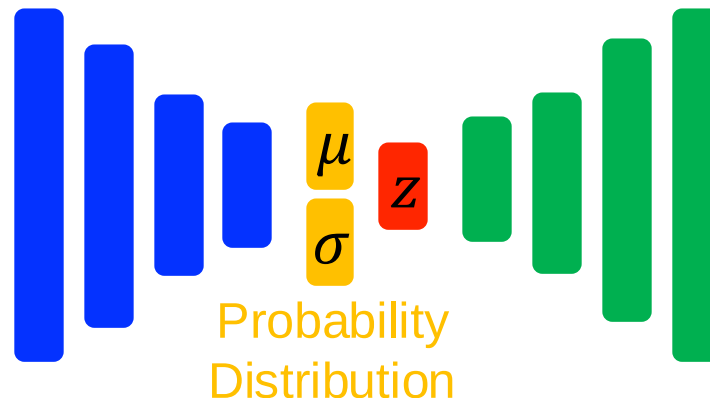
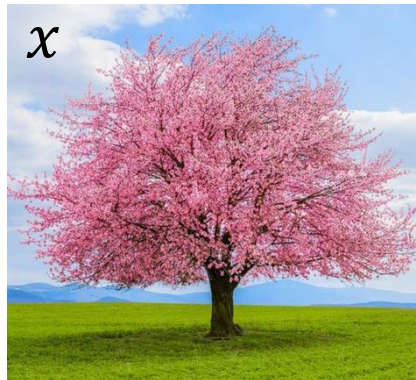
$$\mathcal{L}(\phi, \theta; x, z) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - D_{\text{KL}}(q_\phi(z|x) \parallel p(z))$$

Expectation: we are dealing with random variables

Kullback-Leibler divergence: measure of how one probability distribution is different from another

Reminder: summation or integration of all possible values from a random variable

The **Encoder**
 $q_\phi(z|x)$.



The **Decoder**
 $p_\theta(x|z)$.

Variational Autoencoders (VAE)

[Kingma and
Welling, 2013]



We want the probability distribution to be smooth and not to overfit or memorise any of the latent variables within vector z .

$$\mathcal{L}(\phi, \theta; x, z) = \mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x|z)] - \boxed{D_{\text{KL}}(q_{\phi}(z|x) \parallel p(z))}$$

- By reducing the distance between the probability distribution and a well-behaved prior distribution (such as a *Normal Gaussian*), we:
 - encourage the latent variables to be evenly distributed around the centre of the latent space
 - and deter the model from clustering points in specific regions and cheat by memorising the data.

Variational Autoencoders (VAE)

[Kingma and Welling, 2013]

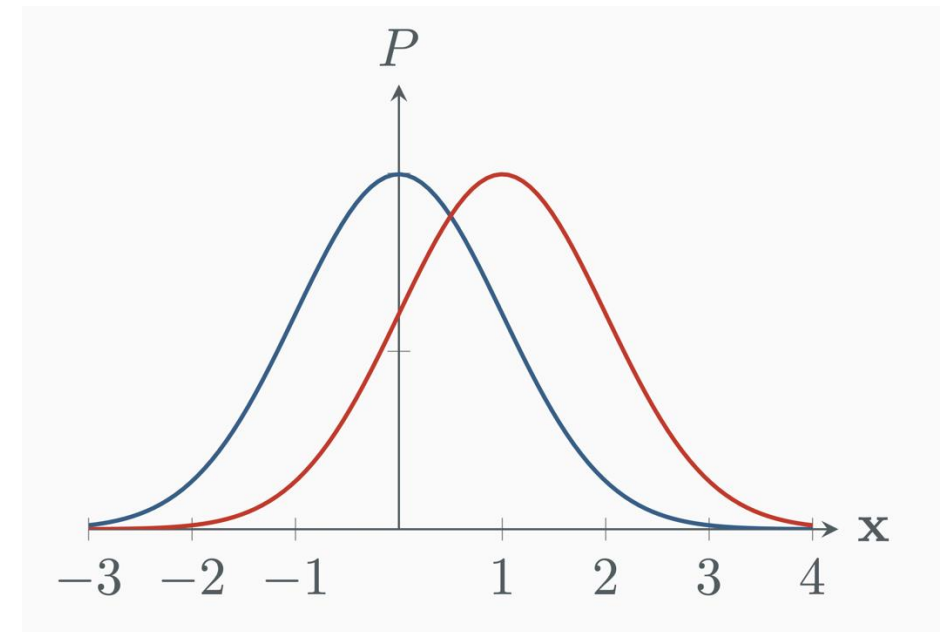


Definition: Kullback–Leibler divergence

The Kullback-Leibler divergence (also called relative entropy) measures the difference between distributions and is asymmetric and non-negative:

$$D_{\text{KL}}(p \parallel q) = \int p(x) \log \left(\frac{p(x)}{q(x)} \right) dx$$

Where $D_{\text{KL}}(p \parallel p) = 0$. Practically, the KL divergence is sensitive at the tails of the distribution.



Desmos Visualisation: <https://www.desmos.com/calculator/2sboqbhler>

Variational Autoencoders (VAE)

[Kingma and
Welling, 2013]



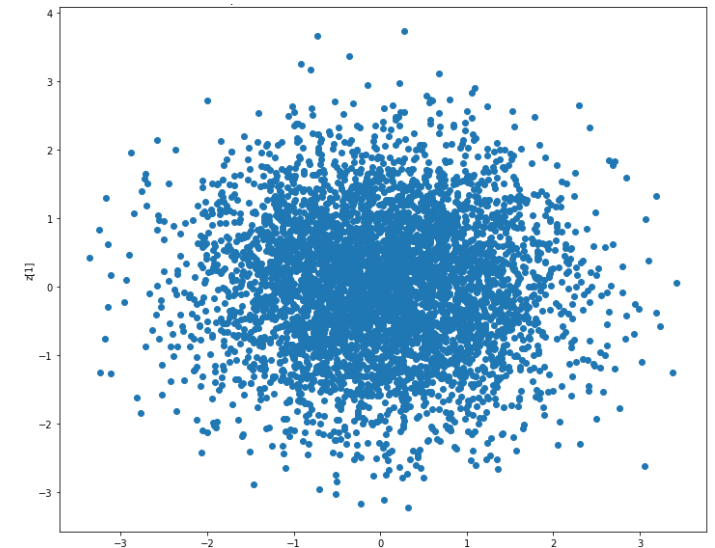
Let's visually inspect how regularisation with the KL term helps.

- We can analyse the effect of the *KL* term by weighting the loss and experimenting with different values:

$$\text{loss} = \text{reconstruction_loss} + c * \text{kl_loss}$$

- We will consider an autoencoder with a latent space of 2 dimensions trained on the MNIST dataset.

For reference, let's look at points sampled from the *standard Gaussian* in two dimensions.



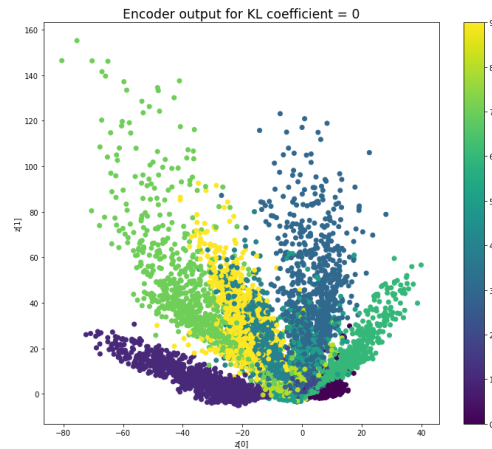
Variational Autoencoders (VAE)

[Kingma and Welling, 2013]

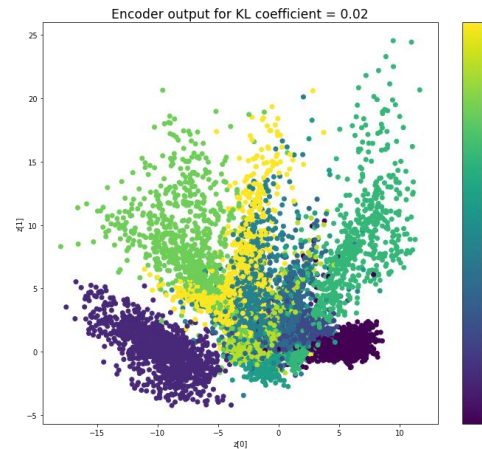


We can see how the points begin to cluster with *no* or *a small* $k1_loss$.

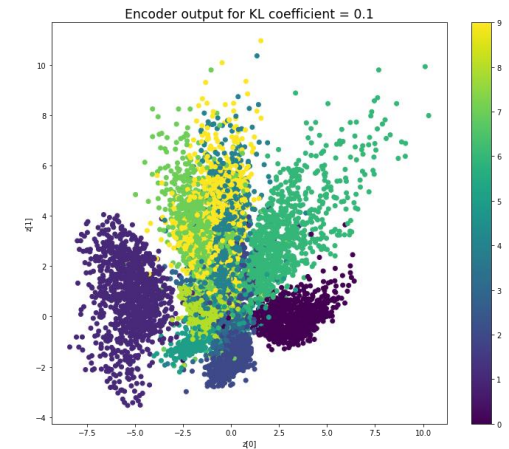
$C = 0.0$



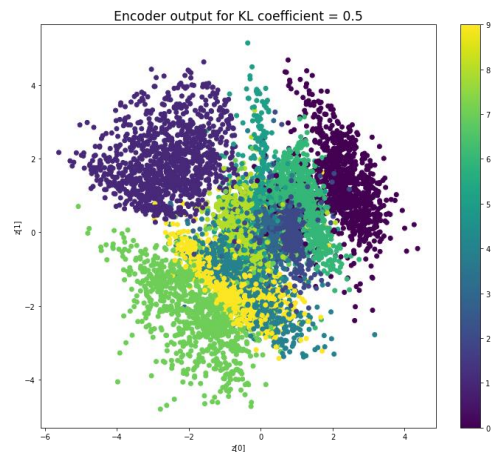
$C = 0.02$



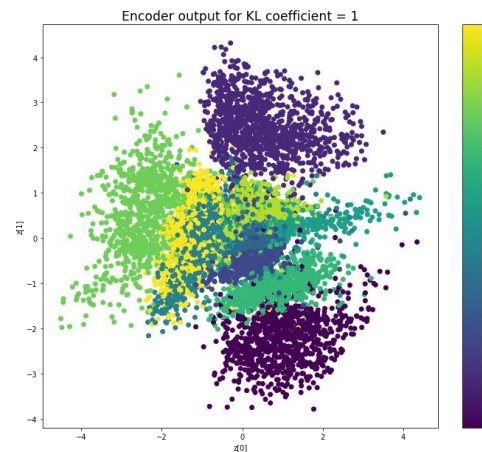
$C = 0.1$



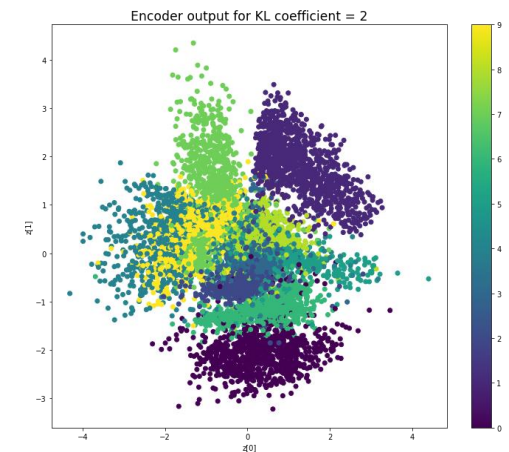
$C = 0.5$



$C = 1.0$



$C = 2.0$



Variational Autoencoders (VAE)

[Kingma and Welling, 2013]

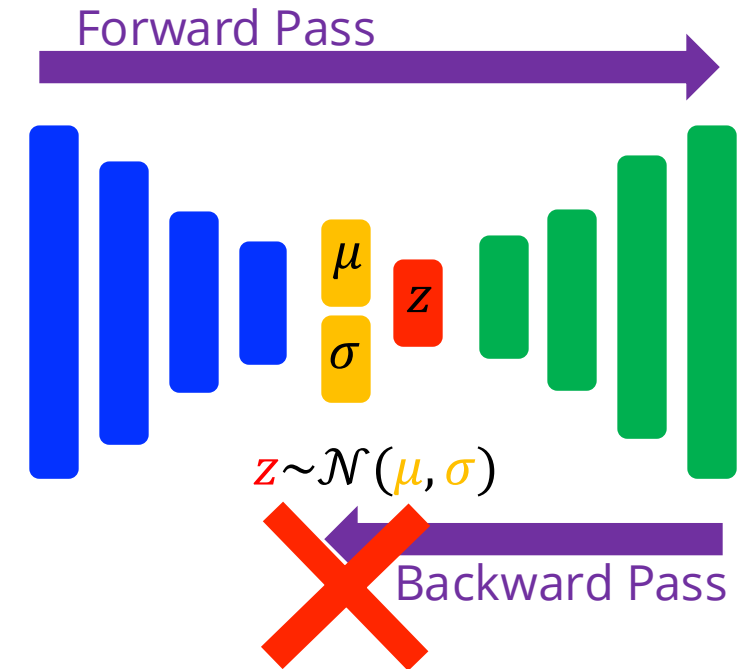


Issue to overcome: backpropagation through a sampling layer

Solution: **Reparameterisation Trick**

$$z = \mu + \sigma \odot \varepsilon \quad \text{where } \varepsilon \sim \mathcal{N}(0,1)$$

z will be a sum of a fixed vector μ and a fixed vector σ scaled by random constants ε sampled from the prior distribution (standard Gaussian).

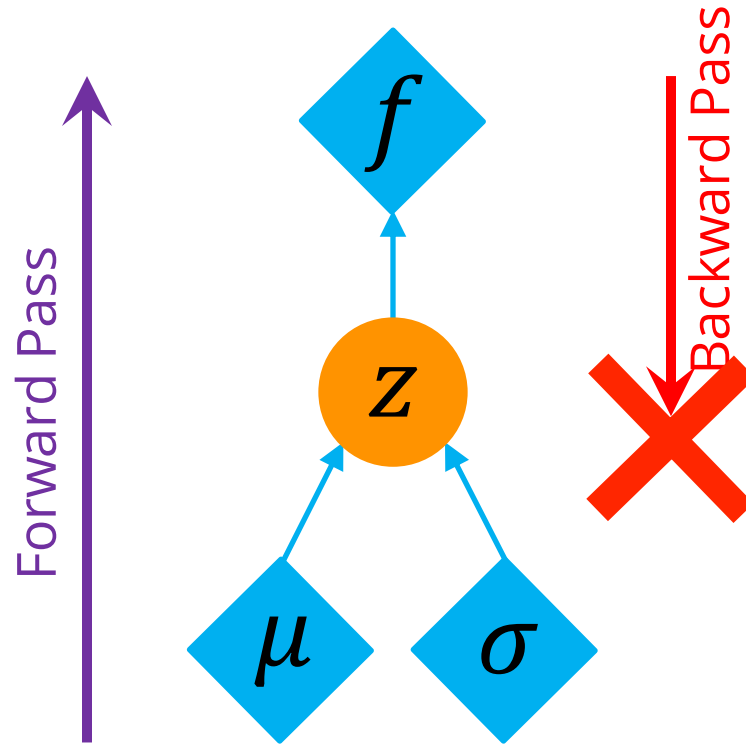


Variational Autoencoders (VAE)

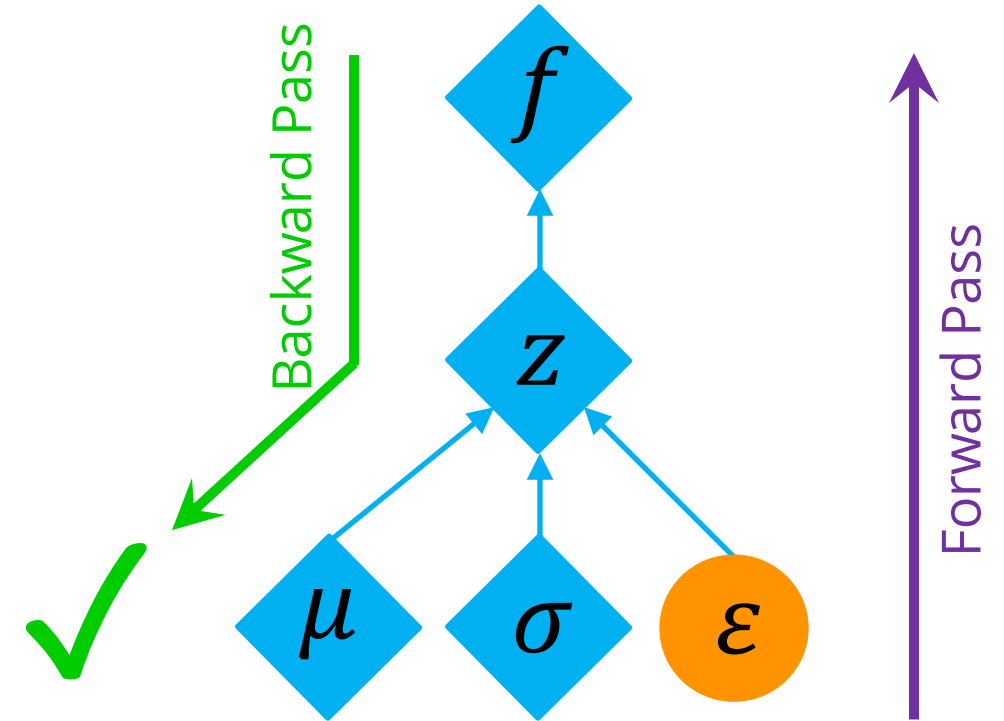
[Kingma and Welling, 2013]



- Reparameterisation Trick



Original Form



Reparameterised Form

Variational Autoencoders (VAE)

[Kingma and Welling, 2013]



Definition: Evidence Lower Bound (ELBO)

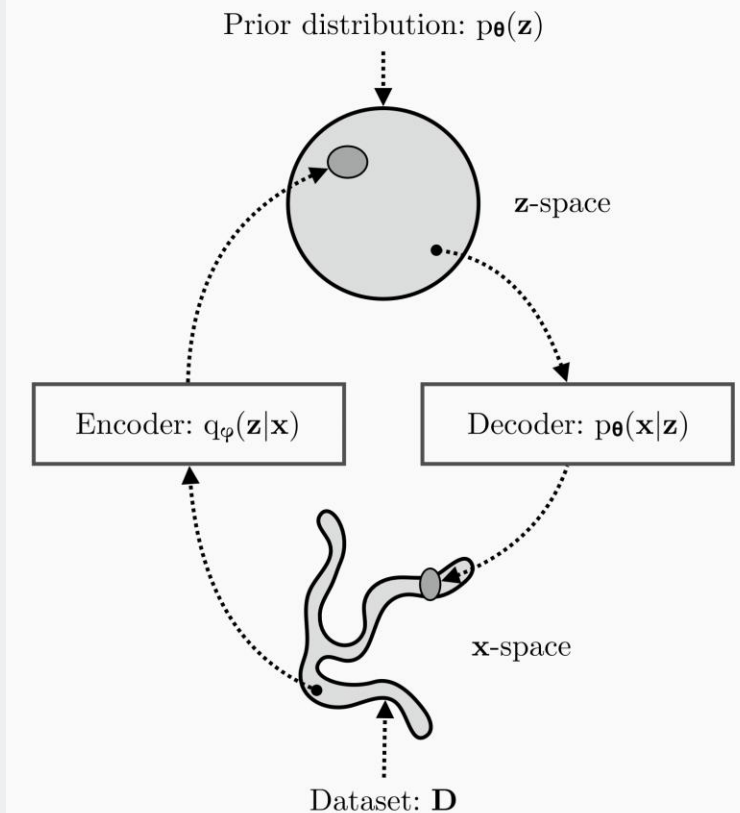
VAEs have three components:

1. The decoder $p_{\theta}(x|z)$
2. The approximate posterior (encoder) $q_{\phi}(z|x)$
3. The prior distribution $p_{\theta}(z)$

They are trained with the reparameterisation trick to maximise the evidence lower bound (ELBO):

$$\log p_{\theta}(x) \geq \mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x|z)] - D_{\text{KL}}(q_{\phi}(z|x) \parallel p(z))$$

There are more advanced and state-of-the-art methods, such as stacking VAEs hierarchically [Child et al., 2021].



Variational Autoencoders (VAE)

[Kingma and Welling, 2013]

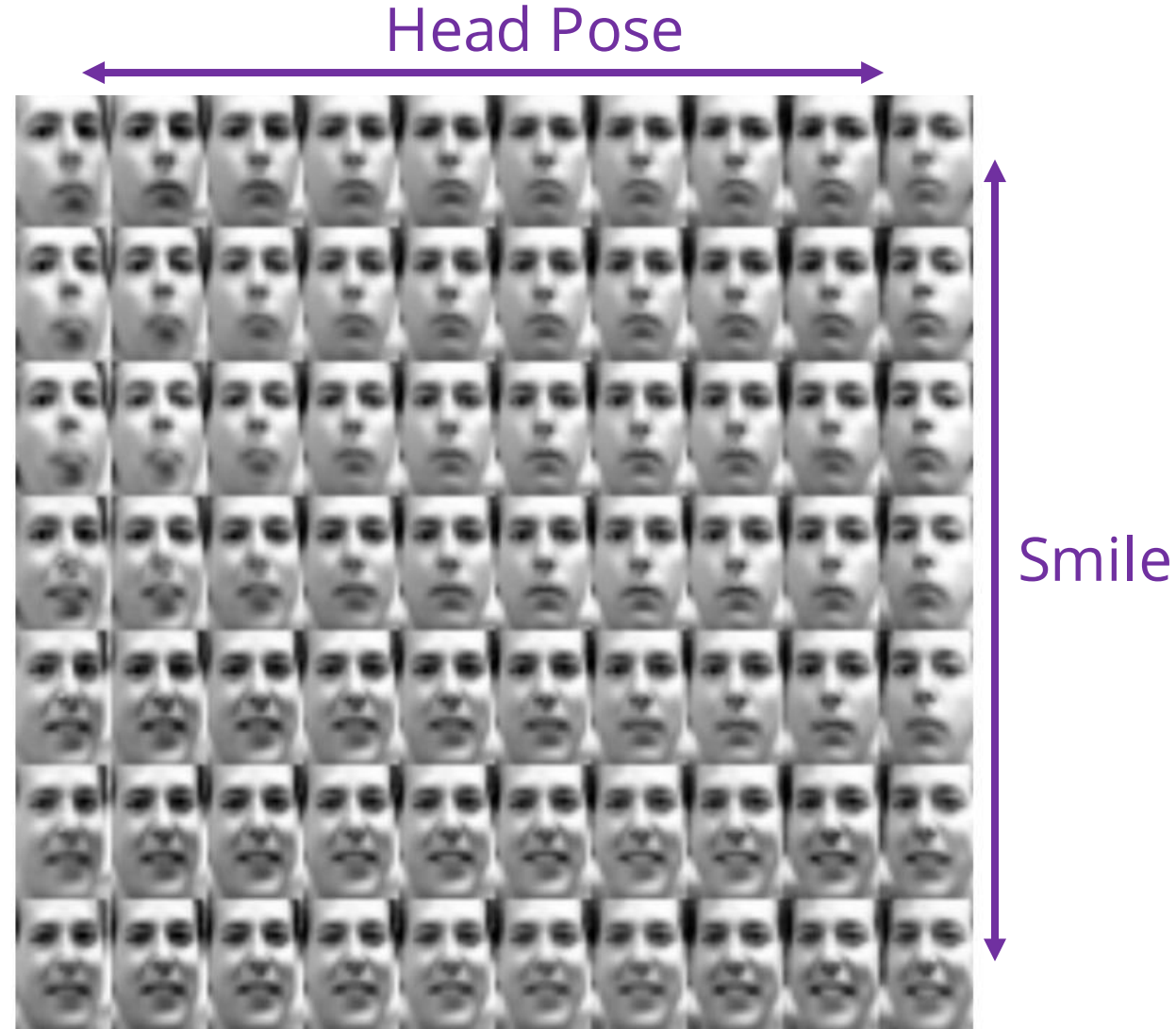


- **Results**

The latent variables are forced to be independent of each other.



Different dimensions of z encode interpretable factors of variation.



Let's Look at Some Code!



Example of a Variational Autoencoder

Code on GitHub: https://github.com/atapour/dl-pytorch/blob/main/VAE_Example/VAE_Example.ipynb

Code on Colab: https://colab.research.google.com/github/atapour/dl-pytorch/blob/main/VAE_Example/VAE_Example.ipynb

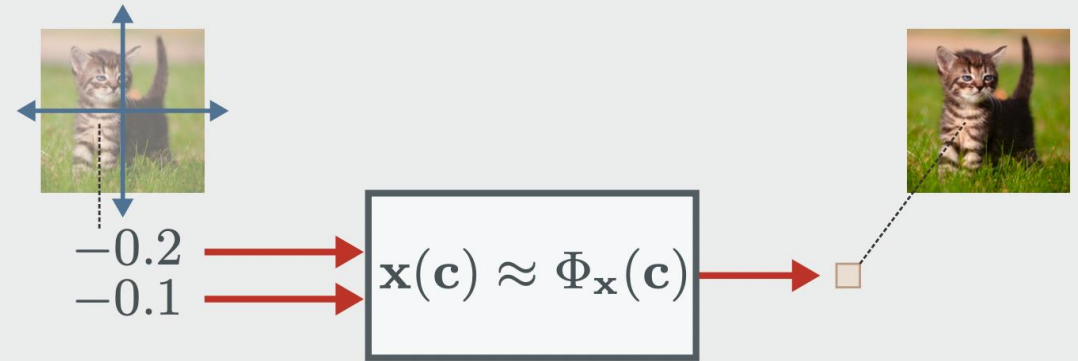
Definition: implicit representations

Consider data $\Phi: \mathbb{R}^m \rightarrow \mathbb{R}^n$, like a single image, as a function of coordinates $\mathbf{c} \in \mathbb{R}^m$. The aim is to learn a neural approximation of Φ that satisfies an implicit equation:

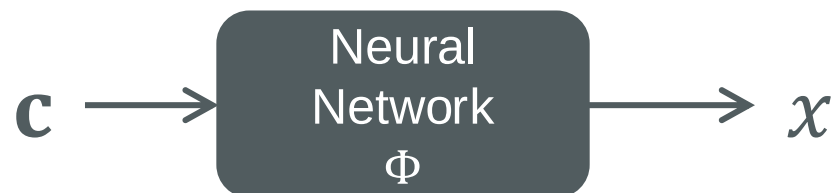
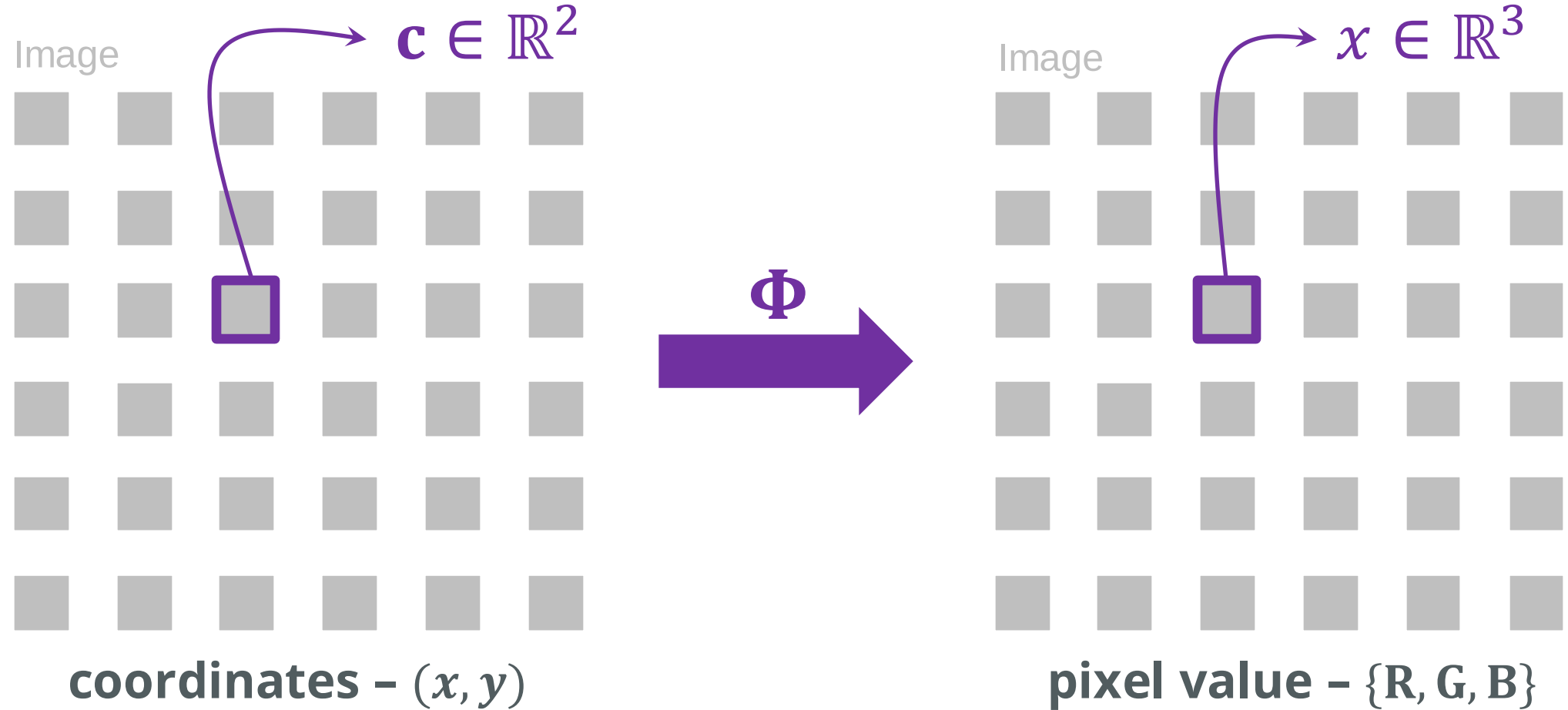
$$R(\mathbf{c}, \Phi, \nabla_{\Phi}, \nabla_{\Phi}^2, \dots) = 0, \Phi: \mathbf{c} \rightarrow \Phi(\mathbf{c})$$

This has applications in 3D modelling, image, video, audio representation.

Example: Implicit Network



Implicit Models



We can have a neural network approximate the image function.

Implicit Representation Networks SIREN

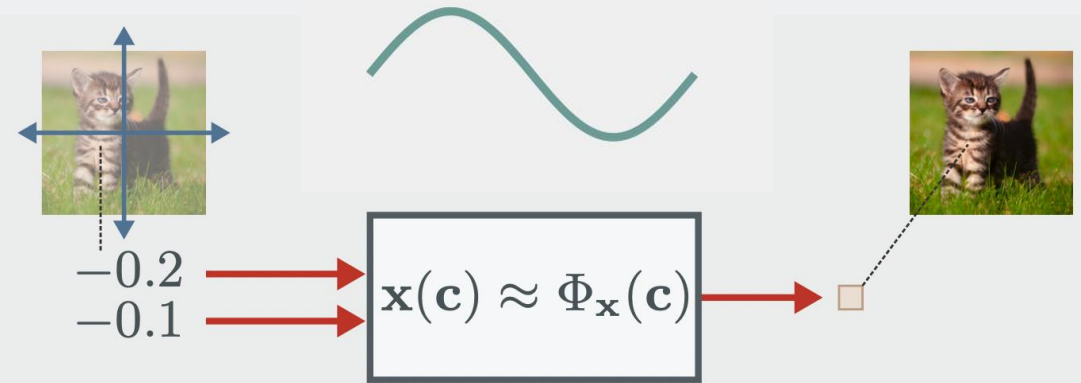


Definition: SIREN

Sinusoidal REpresentation Networks (SIREN) are a simple implicit representation network, but use **sin** (with clever initialisation) as their choice of non-linearity [Sitzmann et al., 2020].

sin is periodic, so it allows to capture patterns over all of the coordinate space (translation invariant, like convolutions).

Example: SIREN



<https://www.vincentsitzmann.com/siren>

Implicit Representation Networks NeRF

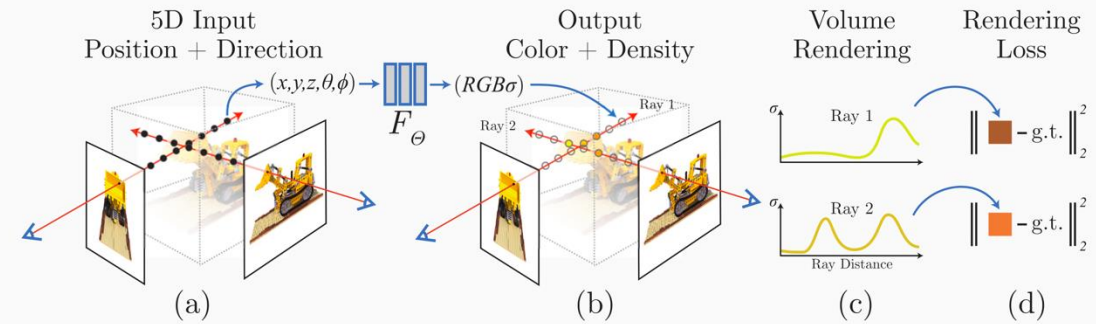


Definition: NeRF

Neural Radiance Fields (NeRF) are similar to SIRENs, but instead of representing an image, they represent a single 3D scene [Mildenhall et al., 2020].

They map from pixel positions (x, y, z) and a viewing direction (θ, ϕ) to a colour and density value σ integrated via a ray on F_θ .

Example: NeRF



<https://www.matthewtancik.com/nerf>

There are other implicit models:

<https://cwkkx.github.io/data/GON/>

What we learned today!



1 Divergence measures

- Kullback–Leibler divergence

2 Variational autoencoders

- Autoencoders
- VAEs
- Reparameterisation trick
- ELBO

3 Implicit networks

- Definition
- SIRENs
- NeRFs